

Organização do Repositório Git

Projeto	Formandos — gestão administrativa de formaturas e eventos
Marco do cronograma	22/09/2026 (Arquitetura inicial + organização do repositório Git)
Versão	1.0 — elaborada em 26/09/2026
Equipe	Luanderson Arlindo, Luiz Orlando, José Renato e Vinícius
Repositório	https://github.com/luandersonarlindo/Formandos

1. Visão geral

Item	Situação
Hospedagem	GitHub, <code>luandersonarlindo/Formandos</code> (origin)
Ramo principal	<code>main</code> (único ramo hoje)
Início do histórico	25/09/2026, com o commit criado pelo <code>create-next-app</code>
Commits até 26/09/2026	32, no padrão <i>Conventional Commits</i>
Licença de uso	Projeto acadêmico

2. Estrutura de pastas

Formandos/ ├─ README.md ├─ AGENTS.md ├─ package.json ├─ db/ ├─ docs/ cronograma ├─┬─ entregas/ ├─ public/ ├─ scripts/ ├─┬─ teste/ ├─ docs/ ├─ src/ ├─ app/ api ├─ actions/ ├─ components/ inicial) ├─ lib/ ├─ proxy.ts	visão geral, tecnologias, rotas, como executar regras para quem programa (ler a documentação do Next.js 16) dependências e scripts (dev, build, typecheck, test, db:*) schema.sql, apply-schema.mjs e seed.mjs (banco de dados) guia de estudo, catálogo de enquetes e entregas do um subdiretório por data do cronograma, com .md e .pdf arquivos estáticos dados de teste, capturas, auditoria e testes de fluxo geração de PDF a partir dos documentos em Markdown rotas: (publico), (onboarding), (app), (admin), (master), Server Actions (mutações), uma por área ui/ (genéricos), features/ (de domínio) e vitrine/ (página consultas, regras, autenticação, e-mail e testes (*.test.ts) checagem rápida de sessão
---	---

3. Estratégia de ramos

Hoje: todo o trabalho é feito em `main`.

Recomendado a partir da Sprint 2, com quatro pessoas:

Ramo	Uso	Exemplo
<code>main</code>	Sempre funcionando; é o que se apresenta	
<code>feat/<assunto></code>	Uma funcionalidade	<code>feat/tema-escuro</code>
<code>fix/<assunto></code>	Uma correção	<code>fix/id-repetido-duvida</code>
<code>docs/<assunto></code>	Documentação	<code>docs/status-report</code>

Fluxo: criar o ramo a partir de `main` → commits pequenos → abrir um **Pull Request** → outra pessoa revisa → *merge* em `main` → apagar o ramo. Assim as quatro pessoas contribuem em paralelo, o histórico mostra quem fez o quê e a `main` não quebra.

4. Convenção de commits

Formato: `tipo(escopo): descrição no imperativo, em português`. O corpo do commit explica o **porquê**.

Tipo	Quando usar	Exemplo do histórico
<code>feat</code>	Funcionalidade nova	<code>feat(votacoes): votação em enquetes com voto identificado e alterável</code>
<code>fix</code>	Correção de erro	<code>fix(duvidas): troca o id do campo de dúvida, que repetia o do conteúdo principal</code>
<code>docs</code>	Documentação	<code>docs: registra o login por email e senha e o envio de email</code>
<code>test</code>	Testes	<code>test: adiciona vitest e testes das funções puras</code>
<code>chore</code>	Manutenção, configuração, scripts	<code>chore(test): adiciona testes de fluxo no navegador</code>

Distribuição do histórico até 26/09/2026: 19 `feat`, 5 `docs`, 5 `chore`, 1 `fix`, 1 `test` e o commit inicial do gerador.

Regras:

1. Um commit = uma ideia. Não misturar correção e funcionalidade nova.
2. Nunca commitar sem rodar `npm run typecheck` e `npm test`.
3. Nunca commitar segredos (`.env.local`, chaves, `client_secret_*.json`).
4. Mensagem clara: quem lê o histórico entende o que mudou e por quê.

5. O que o Git ignora (`.gitignore`)

Padrão	Motivo
<code>/node_modules</code> , <code>/.next</code> , <code>/out</code> , <code>*.tsbuildinfo</code> , <code>next-env.d.ts</code>	Gerados na instalação ou no build
<code>.env*</code> (exceto <code>.env.example</code>)	Segredos locais
<code>client_secret_*.json</code>	Credenciais do Google
<code>scripts/teste/.tmp/</code>	Cookies e capturas dos testes
<code>.vercel</code>	Sobras de hospedagem

O arquivo `.env.example` é o **modelo** das variáveis de ambiente; quem clona copia para `.env.local` e preenche.

6. Como começar (clonar e rodar)

```
git clone https://github.com/luandersonarlindo/Formandos.git
cd Formandos
npm install
cp .env.example .env.local          # preencher DATABASE_URL, BETTER_AUTH_SECRET etc.
npm run db:auth                    # cria as tabelas do Better Auth
npm run db:schema                  # cria as tabelas do domínio
npm run db:seed                    # carrega o catálogo padrão
npm run dev                        # http://localhost:3000
```

Antes de cada commit: `npm run typecheck` e `npm test` .

7. Scripts do `package.json`

Script	O que faz
<code>npm run dev</code>	Servidor de desenvolvimento
<code>npm run build</code> e <code>npm start</code>	Build e execução de produção
<code>npm run typecheck</code>	Gera os tipos das rotas e confere o TypeScript
<code>npm test</code>	Testes unitários (Vitest)
<code>npm run db:auth</code> , <code>db:schema</code> , <code>db:seed</code>	Banco de dados

8. Documentação no repositório

Documento	Onde
Visão geral, tecnologias, rotas e como executar	<code>README.md</code>
Guia de estudo do projeto	<code>docs/guia-de-estudo.md</code>

Documento	Onde
Catálogo de enquetes (fonte do <i>seed</i>)	<code>docs/catalogo-enquetes.md</code>
Entregas do cronograma (<code>.md</code> e <code>.pdf</code>)	<code>docs/entregas/</code>
Como testar as telas	<code>scripts/teste/README.md</code>

9. Situação e recomendações

- **Fato:** até 26/09/2026, todos os commits partem de uma única conta (`luandersonarlindo`), e há um único ramo.
- **Recomendação 1:** cada integrante escolher itens do *Sprint Backlog* e trabalhar em um ramo próprio, com Pull Request.
- **Recomendação 2:** proteger a `main` no GitHub, exigindo um Pull Request aprovado antes do *merge*.
- **Recomendação 3:** usar *Issues* e um quadro do GitHub Projects com as colunas do Sprint Backlog.
- **Recomendação 4:** marcar a entrega de cada sprint com uma *tag* (por exemplo, `sprint-1`).